Gemini CLI Concrete Architecture

Authors:

Riche Cai: 22twq4@queensu.ca

Logan Jarvis: 21lsj2@queensu.ca

Colin Dreany: 22lns3@queensu.ca

Kalan Morris-Poolman: 22lyvd@queensu.ca

2026-03-13

Queen's University - CISC 322

Instructor: Prof. Bram Adams

Contents

1. Introduction and Overview 3
1.1 Overview of Gemini CLI 3
1.2 Purpose of This Report 3
1.3 Scope and Limitations 3
2. Conceptual Architecture & Changes from A1 3
3. Derivation Process 5
4. Concrete Architecture 5
5. Reflexion Analysis 8
6. Use Cases 9
6.1 Use Case 1 Overview: Applying a Specific Model to Gemini CLI 9
6.2 Use Case 1 Process Explanation: Applying a Specific Model to Gemini CLI 9
6.3 Use Case 1 Sequence Diagram: Applying a Specific Model to Gemini CLI 10
6.4 Use Case 2 Overview: Fix Unit Testing 10
6.5 Use Case 2 Process Explanation: Fix Unit Testing 10
6.6 Use Case 2 Sequence Diagram: Fix Unit Testing 11
7. Data Dictionary 12

8. Naming Conventions 12
9. Conclusion 13
10. Lessons Learned 13
11. AI Usage 14
12. References 16

1. Introduction and Overview

1.1 Overview of Gemini CLI

The Gemini CLI is an open-source, command-line interface system that utilizes large language models (LLMs) (Google, 2025). Built around Google's Gemini family of models, the system enables Users to issue natural language commands that can be interpreted, reasoned, and translated into development actions within the User's workspace (Google, 2025). Gemini CLI is primarily intended for developers and technical Users who want to augment their productivity using AI-driven assistance without leaving the terminal environment. Its architecture emphasizes safety, transparency, and extensibility, enabling new tools and integrations to be added over time without disrupting existing functionality.

1.2 Purpose of This Report

The purpose of this report is to analyze and document the concrete architecture of Gemini CLI. The report aims to accurately outline the system's major layers and components, describe their responsibilities, dependencies, and how they interact to support the overall functionality of the software. Additionally, this report serves as an academic exercise for CISC 322, applying architectural concepts to a real-world, open-source system.

1.3 Scope and Limitations

The scope of this report encompasses the high-level concrete architecture of the Gemini CLI, structured as a box-and-arrow diagram to illustrate subsystems and their dependencies. Following a rigorous derivation process, we transitioned from the A1 conceptual architecture by iteratively mapping source files and folders to specific components. This analysis includes a detailed decomposition of our previous interpretations of the system layers into their true components, exploring both the new conceptual roles and their concrete implementations. Two primary use cases are modeled via sequence diagrams, with method-level edges mapped to concrete calls within the source code. A reflexion analysis that discusses discrepancies between our conceptual model and the actual system as evidenced by git history and code structure, concluding with specific lessons learned regarding the project's architectural evolution.

2. Conceptual Architecture & Changes from A1

In Assignment 1, we proposed a conceptual architecture for Gemini CLI that described the system at a high level. The goal of this architecture was to explain the major components of the system and how they interact, without focusing on specific implementation details. At a conceptual level, Gemini CLI was modeled as a layered command-line AI assistant consisting of a user-facing interface, a central orchestration component, and core layers responsible for executing tools and interacting with external services (Jalateras, 2024).

Figure 0: Conceptual Architecture Box-and-Line Diagram

Figure 0: Conceptual Architecture Box-and-Line Diagram

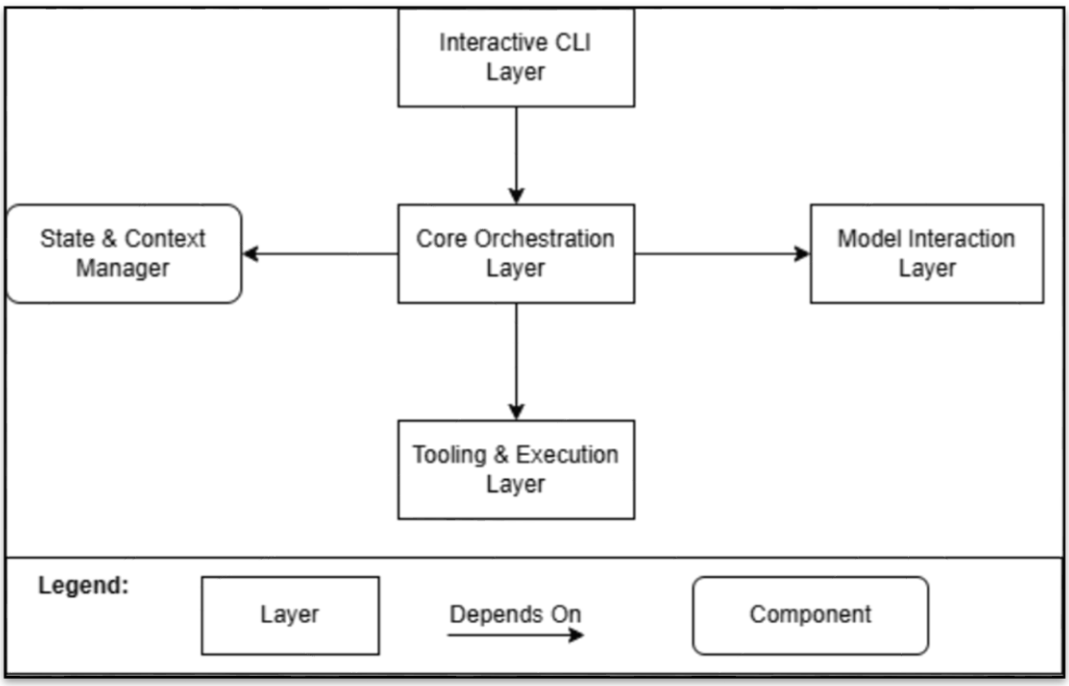

Figure 0: Conceptual Architecture Box-and-Line Diagram (Prior to adjustment)

In this model, users interact with the system through the Terminal Interface, which sends requests to a central Orchestration Engine. The Orchestration Engine then coordinates communication with the Gemini API and determines whether additional actions such as tool execution or task delegation are required.

After analyzing the Gemini CLI source code using the Understand static analysis tool, we refined our conceptual architecture to better match how the system is actually implemented. While the general structure from Assignment 1 remained largely correct, we found that several responsibilities were split across more clearly defined subsystems in the real codebase.

For example, the component we originally described as the Core Orchestration Layer is actually implemented through multiple modules including core, commands, routing, and services.
In the conceptual architecture this subsystem appeared as the Core Orchestration Layer, but repository analysis revealed that it is implemented through several modules including core, commands, routing, and services. Together, these modules form what we now describe as the Orchestration Engine. Similarly, the command-line interaction logic is implemented in the cli package, which we now explicitly refer to as the Terminal Interface.

In addition, our analysis revealed several subsystems that were not clearly separated in our original conceptual architecture. These include the Configuration System (which manages runtime settings and contextual information), the Security & Execution Environment (which enforces safety policies for tool execution), the External Integration Layer (which connects the system to external Model Context Protocol (MCP) servers), and lastly the Task Delegation System (which enables distributed task execution through the A2A framework).

These components became clear after examining the subsystem grouping and directory structure extracted from Understand. Each architectural subsystem corresponds closely to specific

directories within the repository, allowing us to refine our conceptual architecture so that it more accurately reflects the structure of the real system.

Overall, the main idea behind the architecture did not change from Assignment 1. The system still follows a layered design where the Terminal Interface handles user interaction, the Orchestration Engine coordinates the main workflow, and several supporting subsystems provide services such as tool execution, task delegation, configuration management, and security enforcement. The main difference is that the updated conceptual architecture separates these responsibilities more clearly based on how they are implemented in the codebase.

3. Derivation Process

To begin deriving the architecture of GeminiCLI, we first became familiar with the provided software, Understand, by following the provided tutorial (SciTools, 2024). We then set up a dependency graph based on the directory structure to get a better idea of how GeminiCLI was organized (SciTools, 2024). From this graph, we were able to see that the system was organized around a central packages/ directory containing cli, core, a2a-server, vscode-ide-companion, and test-utils, alongside some supporting infrastructure in evals, integration-tests, and scripts. To get a clearer picture of the structure, we went into core/src/, which revealed many of the necessary files for some of the components we had identified in A1, and helped us identify components we were missing from the Global Feedback about A1 document. We also inspected the repository's git history to better understand how certain modules evolved over time and to confirm whether specific responsibilities were implemented as standalone components or integrated into other subsystems. From our original components in A1, the Interactive CLI (now Terminal Interface), Core Orchestration Layer (now Orchestration Engine), and Tooling & Execution Layer (Tool Execution Framework) were all present, while the other components we identified were either not standalone components themselves or were absorbed into a new component. The new components from the Global Feedback about the A1 document that we added, the External Integration Layer, Task Delegation System, Security & Execution Environment, and Configuration System, were also present. We then assigned each file and folder to its corresponding: packages/cli mapped to the Terminal Interface; the Orchestration Engine mapped to core/, commands/, services/, and routing/; the Tool Execution Framework mapped to tools/ and scheduler/; the External Integration Layer mapped to mcp/; the Task Delegation System mapped to agents/ and packages/a2a-server; the Security & Execution Environment mapped to safety/ and policy/; and the Configuration System mapped directly to config/. Other submodules were also present but were excluded because they were either part of the testing infrastructure or small utility modules. From here, we were able to construct a new dependency diagram based on our new understanding of GeminiCLI's structure and found new dependencies and changes to existing ones from A1.

4. Concrete Architecture

The concrete architecture of Gemini CLI represents how the system is implemented in the source code. To derive this architecture, we analyzed the Gemini CLI repository using the Understand static analysis tool, which allowed us to generate dependency graphs and examine relationships between different modules in the system.

The diagram below shows the subsystem dependency graph generated using Understand. In this diagram, each box represents a subsystem, while the arrows represent dependencies between them. These dependencies may correspond to function calls, shared references, or other interactions between modules.

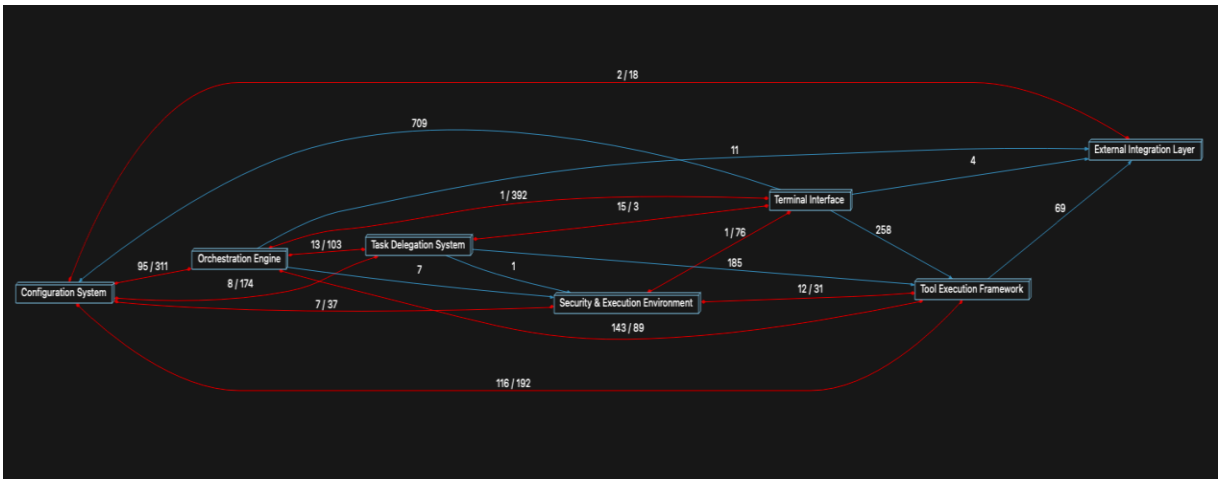

Figure 1: Gemini CLI Box-Line & Dependency Graph

From this analysis, we identified seven main subsystems that together make up the architecture of Gemini CLI. These subsystems were determined by grouping directories and modules based on their responsibilities within the system.

Terminal Interface:

The Terminal Interface is the main entry point for users interacting with Gemini CLI. This component is implemented in the cli directory and is responsible for handling command-line input, displaying output, and managing the overall user interaction flow.

When a user enters a prompt or command, the Terminal Interface parses the input and forwards the request to the Orchestration Engine. It also handles things like confirmation prompts when tools are about to perform potentially sensitive actions and formats the responses returned by the system. Because its main responsibility is user interaction, the Terminal Interface remains relatively independent from the rest of the system's internal logic.

The Terminal Interface interacts with the Orchestration Engine, Security and Execution Environment, Configuration System, Tool Execution Framework, and Task Delegation System.

Orchestration Engine:

The Orchestration Engine acts as the central coordinator of the system. It is implemented across several modules, including core, commands, routing, and services. This subsystem is responsible for interpreting user requests and determining how the system should respond. In particular, it handles tasks such as constructing requests to the Gemini API, maintaining conversation context, routing commands to the appropriate subsystem, and coordinating tool execution when necessary.

As shown in the dependency graph, the Orchestration Engine interacts with several other subsystems, including the Tool Execution Framework, Task Delegation System, Configuration System, Terminal Interface, and External Integration Layer.

Tool Execution Framework:

The Tool Execution Framework enables Gemini CLI to perform actions within the user's workspace. This subsystem is implemented in the tools and scheduler directories. It manages the discovery, scheduling, and execution of tools that may be triggered by the language model. These tools can perform operations such as reading files, modifying code, or running development commands.

Tool Execution Framework interacts with the Configuration System, External Integration Layer, Orchestration Engine, Task Delegation System, and Security and Execution Environment.

Task Delegation System:

The Task Delegation System coordinates and executes complex tasks that require specialized agents. It is responsible for delegating tasks to subagents, managing agent-to-agent communication, and handling the results returned from those agents. It achieves this through the agent's module in core, which handles the internal coordination logic and the a2a-server package, which enables remote agent communication.

The Task Delegation System interacts with the Configuration System, Orchestration Engine, Security and Execution Environment, and Tool Execution Framework.

Configuration System:

The Configuration System manages configuration settings used throughout the system. This subsystem is implemented in the config module. Configuration settings include model parameters, tool settings, and other contextual information required during execution. Because many components depend on these settings, the Configuration System serves as shared infrastructure across multiple parts of the architecture.

The Configuration System interacts with the External Integration Layer, Tool Execution Framework, Security and Execution Environment, Orchestration Engine, and Task delegation System.

Security and Execution Environment:

The Security and Execution Environment is responsible for ensuring that tool execution and system operations are performed safely and within the limits of what has been permitted by the system. This is accomplished through both the safety module in core, which ensures that tools run in an isolated environment where they cannot perform unauthorized operations on the user's system, and the policy module in core, which enforces permission rules that determine which actions are permitted based on the current execution context.

The Security and Execution Environment interacts with the Terminal Interface, Tool Execution Framework, and Configuration System.

External Integration Layer:

The External Integration Layer manages communication with external tools and services through the Model Context Protocol (MCP) (Google, 2025). This subsystem is implemented using the mcp module within core, which is responsible for discovering, connecting to, and invoking tools provided by external MCP servers. By doing so, it effectively extends the capabilities of the Gemini CLI beyond its built-in tools, such as file system operations, shell commands, and web search. Having it as its own module keeps connections to outside tools and services separate from the rest of the system, making it easy to add new MCP servers without changing other components.

The External Integration Layer only interacts with the Configuration System.

5. Reflexion Analysis

The conceptual architecture of the Gemini CLI was designed around a five-component layered structure with clear separation between the Interactive CLI Layer, Core Orchestration Layer, State and Context Management, Model Interaction Layer, and Tooling and Execution Layer components. In comparison to the concrete architecture, there are quite a few discrepancies. These divergences were confirmed through repository inspection and git history analysis of the relevant modules.

Concerning the conceptual architecture, three of the old components, the Interactive CLI Layer (now Terminal Interface), Core Orchestration Layer (now Orchestration Engine) and Tooling and Execution Layer (now Tool Execution Framework), were renamed into components listed in the provided Conceptual Architecture Summary, some of which expanded on the functionality that we believed they were responsible for. The State and Context Manager no longer became an individual component in the concrete architecture, but instead, its responsibilities were absorbed by the Orchestration Engine and a new component, the Configuration System. This makes sense as configuration settings are fixed values shared across the system, while context and state change dynamically during execution and are better managed within the orchestration logic. The Model Interaction Layer (Gemini API) was removed entirely, as it became an external dependency since it is simply an API that the Orchestration Engine calls out to.

Regarding the concrete architecture, four additional components were added based on the provided Conceptual Architecture Summary: Configuration System, Security & Execution Environment, External Integration Layer, and Task Delegation System. The Configuration System was previously folded into the Core Orchestration Layer in A1, but the config module proved to be a distinct and self-contained module with its own responsibilities. The Security and Execution Environment was overlooked in A1, with both the safety and policy modules showing it existed as a dedicated component. The External Integration Layer was missed in A1 because MCP integration was not identified when reading the documentation for GeminiCLI. The Task Delegation System was also entirely absent from A1, but the agents module and the top-level a2a-server package confirm that agent delegation is a core part of the system.

In terms of dependencies, we discovered many more than we thought between both the components we had correctly identified and the new ones. For each, the dependencies are as follows:

- Terminal Interface: Orchestration Engine, Security and Execution Environment, Configuration System, Tool Execution Framework, and Task Delegation System.
- Orchestration Engine: Tool Execution Framework, Task Delegation System, Configuration System, Terminal Interface, External Integration Layer, and Security and Execution Environment.
- Tool Execution Framework: Configuration System, External Integration Layer, Orchestration Engine, Task Delegation System, and Security and Execution Environment.
- Task Delegation System: Configuration System, Orchestration Engine, Security and Execution Environment, and Tool Execution Framework.
- Configuration System: External Integration Layer, Tool Execution Framework, Security and Execution Environment, Orchestration Engine, and Task Delegation System.
- Security and Execution Environment: Terminal Interface, Tool Execution Framework, Configuration System.
- External Integration Layer: Configuration System.

The differences between the conceptual and concrete architectures show that, while some of what we did in the conceptual architecture was correct, we underestimated the system's internal complexity. As a result, the concrete architecture revealed a more interconnected system with many more dependencies between components.

6. Use Cases

6.1 Use Case 1 Overview: Applying a Specific Model to Gemini CLI

This use case models how a User configures the Gemini CLI to use a specific language model for the current session. Instead of selecting a model during each interaction, the User modifies the default model setting through the system's configuration workflow. The goal of this interaction is to demonstrate how Gemini CLI manages internal configuration settings and how the system updates the active model used by the language model integration layer.

6.2 Use Case 1 Process Explanation: Applying a Specific Model to Gemini CLI

The user sends the slash command "/model" to the terminal. The terminal interface interprets this input as a command rather than a prompt. In response, the interface initiates the openModelDialog() workflow, which allows the user to select a language model.

When this workflow begins, the terminal queries the configuration system to determine the model that is currently set for the session. After retrieving this information, the terminal displays a list of available model options for the user.

The user then selects a model by entering the corresponding number associated with a radio button option. Once the selection is made, the terminal calls the configuration system to update the stored model setting.

The configuration system saves the new model and returns confirmation to the terminal interface. The terminal then closes the model selection dialog, completing the interaction and updating the model that will be used for the current session.

6.3 Use Case 1 Sequence Diagram: Applying a Specific Model to Gemini CLI

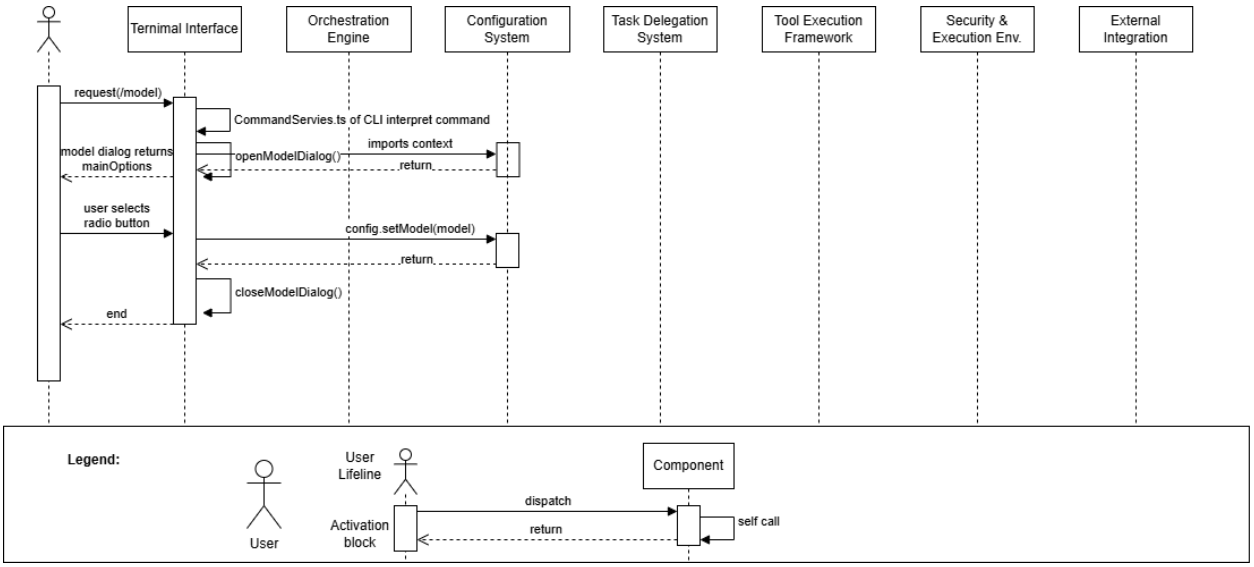

Figure 2: Use Case 1 Sequence Diagram

6.4 Use Case 2 Overview: Fix Unit Testing

This use case models how a Developer uses Gemini CLI to generate a unit test file for an existing source file within a project (Google, 2025). The Developer specifies a source file, such as x.py, and requests that Gemini CLI create a corresponding test suite that validates the functionality of the code and places it in a specific folder. The goal of this interaction is to showcase the user-model interaction workflow of the program.

6.5 Use Case 2 Process Explanation: Fix Unit Testing

The user sends a textual request to the Terminal Interface. The interface interprets the request and determines that it is a Gemini prompt rather than a slash command. The interface then uses handleAtCommand to query the Configuration System for contextual information such as relevant files, resources, and environment data. This context is returned to the interface.

Next, the Terminal Interface retrieves a Gemini client through the Configuration System, which triggers the initialization of the Orchestration Engine. Once the client is available, the terminal sends the request to the engine using sendMessageStream.

The Orchestration Engine then performs prompt preparation using information from the Configuration System. After preparing the request, the engine activates the External Integration component and sends the query to Gemini.

Gemini processes the request and returns a response that includes a request to execute a tool. The engine forwards this information to the interface so the user can view the proposed action. Assuming the user approves the action, the interface triggers a turn execution within the Orchestration Engine.

The engine then dispatches the request to the Tool Execution Framework, which performs the required operations. Once the tool execution is completed, the results are returned to both the engine and the interface.

The interface records and logs the execution outcomes, and then sends the tool results back to Gemini for further analysis. The Orchestration Engine again prepares the prompt and forwards the updated request to Gemini.

Finally, Gemini returns a concluding response describing the actions it performed and the resulting outcome. This response is returned to the Orchestration Engine and then to the Terminal Interface, which outputs the final message to the user and concludes the interaction.

6.6 Use Case 2 Sequence Diagram: Fix Unit Testing

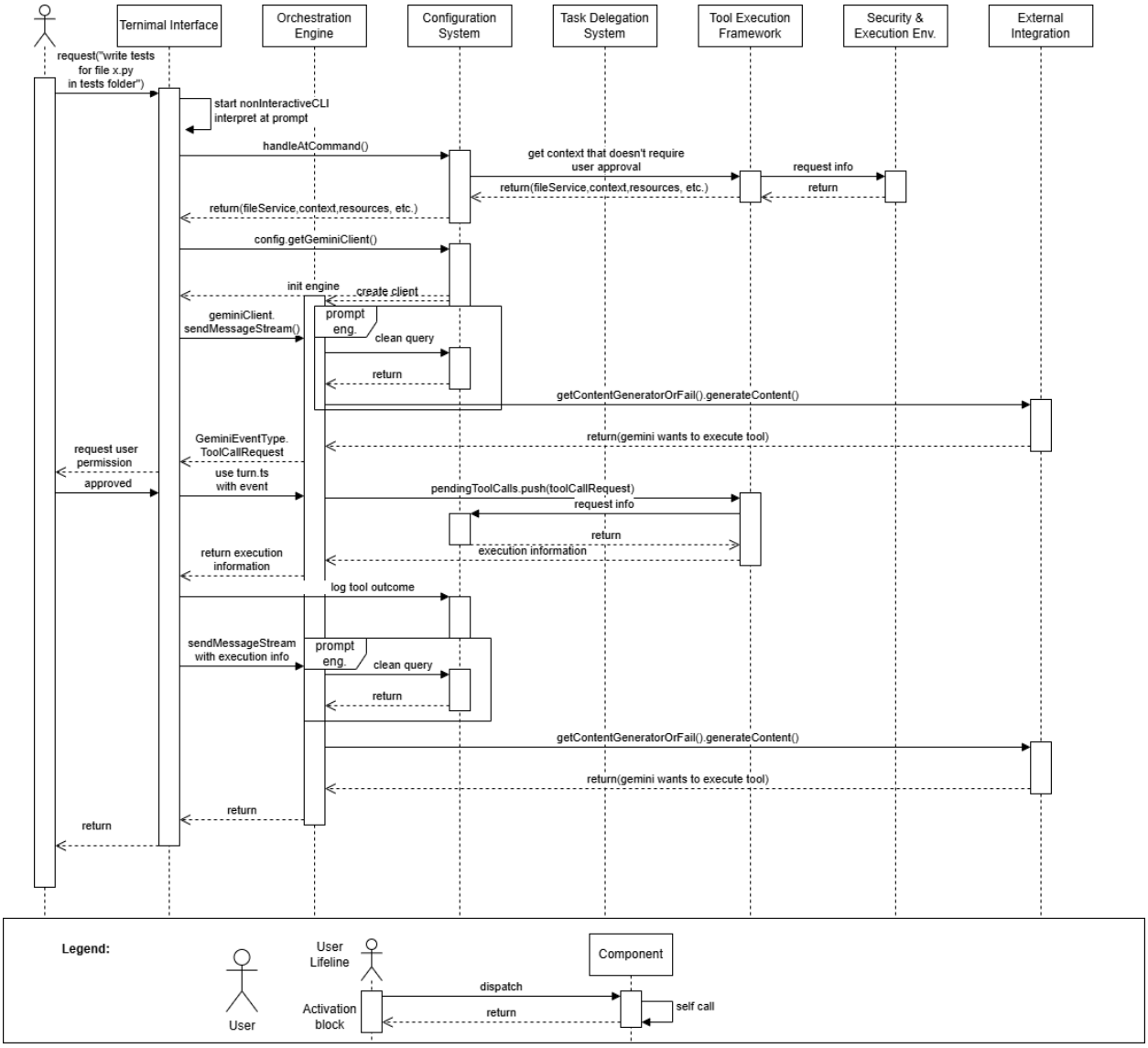

Figure 3: Use Case 2 Sequence Diagram

7. Data Dictionary

This section defines the key architectural terms used throughout this report.

Agent – A specialized execution unit used by the Task Delegation System to perform subtasks or coordinate complex operations.

Configuration System – A subsystem responsible for storing and retrieving runtime configuration settings such as model selection and tool configuration.

External Integration Layer – A subsystem responsible for connecting Gemini CLI to external tools and services through the Model Context Protocol (MCP).

Orchestration Engine – The central subsystem responsible for coordinating system behavior, including processing prompts and routing tasks to other subsystems.

Prompt – A natural language request provided by the user that is interpreted by the language model.

Security and Execution Environment – A subsystem that enforces safety policies and ensures that tool execution occurs within permitted system boundaries.

Task Delegation System – A subsystem responsible for distributing complex tasks to specialized agents.

Terminal Interface – The command-line interface that accepts user input and displays system output.

Tool Execution Framework – The subsystem responsible for discovering, scheduling, and executing tools requested by the system.

8. Naming Conventions

The Gemini CLI repository follows several naming conventions for organizing code and architectural components.

Classes and Components (PascalCase) – Major classes and architectural components are named using PascalCase, where each word begins with a capital letter. Examples include Terminal Interface, Orchestration Engine, and Tool Execution Framework.

Packages and Directories (lowercase) – Repository directories and packages typically use lowercase names to represent modules within the system. Examples include packages/cli, core/config, and core/tools.

Functions and Methods (camelCase) – Functions and methods use camelCase naming, where the first word begins with a lowercase letter and subsequent words begin with uppercase letters. Examples include openModelDialog(), sendMessageStream(), and handleAtCommand().

Commands (Slash Commands) – Commands entered in the CLI begin with a forward slash and trigger specific workflows in the system. Examples include /model, /help, and /tools.

Abbreviations – The report uses several common abbreviations:
API – Application Programming Interface
CLI – Command Line Interface
LLM – Large Language Model
MCP – Model Context Protocol
JSON – JavaScript Object Notation
A2A – Agent-to-Agent

9. Conclusion

This report derived the concrete architecture of the Gemini CLI by analyzing its repository structure and subsystem dependencies (Google, 2025. Using the Understand software, multiple conceptual architectures, and general repository exploration, we identified seven major subsystems in the system: the Terminal Interface, Orchestration Engine, Tool Execution Framework, Task Delegation System, Configuration System, Security and Execution Environment, and the External Integration Layer.

Comparing the conceptual architecture from Assignment 1 with the concrete architecture showed several differences. While the overall layered structure remained mostly consistent, the real implementation distributes responsibilities across more subsystems than we initially expected. Components such as the Configuration System, Security and Execution Environment, External Integration Layer, and Task Delegation System were not clearly separated in the conceptual model but appeared as distinct modules in the codebase.

The use case analysis also demonstrated how these subsystems interact during real workflows. The model selection use case showed direct interaction between the Terminal Interface and Configuration System, while the unit test generation use case demonstrated the standard pipeline involving the Orchestration Engine, tool execution, and external model interaction.

Overall, this project showed how architectural understanding changes when moving from a conceptual design to the real structure of a software system.

10. Lessons Learned

This project highlighted the differences between conceptual and concrete architectures. Our conceptual model captured the main idea of the system, but the concrete architecture showed that responsibilities were split across more subsystems than expected. Real implementations often distribute logic across multiple modules rather than following a clean layered structure.

Using the Understand tool was helpful for identifying subsystem dependencies, but it also produced large dependency graphs that required careful interpretation. Setting up the tool correctly early in the project was important, since troubleshooting issues later would have slowed down the analysis.

Another observation was that not all system interactions pass through the Orchestration Engine. Some actions, such as configuration updates, occur directly between the Terminal Interface and the Configuration System. This showed that real systems sometimes bypass central components for simpler operations.

Finally, the project improved our understanding of how to analyze large codebases. By combining repository exploration, static analysis tools, and reflexion analysis, we were able to move from a conceptual model to a concrete architectural representation of the system.

11. AI Usage

AI Member Profile and Selection Process

For Assignment 2, our team selected to stay with OpenAI GPT-5 (March 2026 version) as our virtual AI teammate. There were many different factors that contributed to this, some shared from assignment 1, but some new that we discovered as we worked on Assignment 2.

Firstly, we retraced our steps to attempt to recreate our successful selection process from assignment 1. We considered the three major AI companions of Claude, ChatGPT, and Gemini; by first considering aspects we want to prioritize and then weighing each AI's pros and cons against each other with the new lens of Assignment 2. After some discussion, we realized that our main priority was still compatibility/cooperability, coming even before sheer model power. Followed by these two was a new idea prompted by our use of AI from Assignment 1: idea creation and brainstorming capabilities. We realized that the majority of our use of our AI companion revolved around generating ideas or helping guide our own ideas.

With these three main goals in mind, we quickly went through each AI companion option. ChatGPT was the natural initial frontrunner due to a clear advantage in team cooperability via the use of group chats, along with our prior experience using it in Assignment 1. After our use of ChatGPT, we figured out that it was also the strongest at idea fuel and creation out of all the competition. Despite this, we still decided to look at the others. Claude offered very strong coding capabilities, but that just was not useful in our goals to help understand and build concrete architecture. Similarly, Gemini was much stronger at large, complex multi-modal tasks, but such tasks would be minimally used by our group, especially when we looked back at our AI usage from Assignment 1 which mainly consisted of fuelling ideas. With these reasons in mind, it was even more evident than in Assignment 1 that OpenAI's GPT-5 was the best option by a landslide for our group. It offered too much compatibility and ease of use, and whose current model aligned with our AI usage goals perfectly.

Tasks Assigned to the AI Teammate

Template Creation

From a successful usage in Assignment 1, we decided to get our AI companion to help create and format a word template for us to follow based on our rubric and the appendixes. This is a task that would otherwise be time consuming and error-prone if done by a person.

Organization and delegation of tasks

Another successful task from Assignment 1, we decided to reuse our AI teammate to

organize what tasks had to be done, and how we could divide it amongst ourselves. AI is much more suited for this than a person. AI can act without individual bias for team members and has a better idea of how intensive sections would be prior to our knowledge.

Mapping Source Directories to Architecture Components

New to Assignment 2, we were having trouble trying to decide how to map folders to appropriate subsystems. Although some of these were easy for us to do, there were a lot of source directories, many of them being quite vague in nature. Due to an AI's large knowledge base, it became a crucial help in trying to decide what belonged in certain subdirectories.

Review, format, and editing

Taking advantage of AI's ability to store large amounts of data easily, we decided again that it would be wise to utilize this to help the AI detect inconsistencies or flaws in our report draft by reading it over, and then comparing it to the appropriate rubric and appendixes.

Interaction Protocol and Prompting Strategy

Once again, we decided to prompt our AI teammate as a collective group. This allows us to work out any possible inconsistencies or overlap with different prompts. Identical to Assignment 1, we started each prompt by one group member having a brief discussion with related teammates about an idea for a prompt they had. This discussion would decide how to design and engineer the prompt to try to get the most desirable outcome using our AI group chat. A key prompt we used in this assignment was "We are analyzing the Gemini CLI repository with the attached directories. Based on typical software architecture patterns, suggest how these modules could be grouped into architectural subsystems and describe the likely responsibility of each subsystem. Use logical justification." We further learned that giving the AI proper context is more important than the prompt itself. By using our ChatGPT group chat, it was able to use its message history and use prior information that would further enhance the response we were looking for. This allowed our prompting strategy to evolve beyond assignment 1 and emphasised the importance of our group chat.

Validation and Quality Control Procedures

Our validation process and quality control procedures were equally as intensive and efficient as before, being very careful of critical mistakes AI is known to make. Despite only having the AI help us in idea generation and guidance without doing any actual work, we made sure to cross-check it. We were simply able to look at the attached documents used (rubric, appendixes, course material, etc.), to make sure the suggestions the AI made were consistent with the source material. If the AI was unable to pass our checks or justify its answers using proper sources, the prompt would be wholly voided and a new one would be created to attempt again, reusing all of our source material.

Quantitative Contribution to Final Deliverable

We estimate that due to increased efficiency in our AI usage, our AI companion contributed roughly 30% of the work put into drafting support for our deliverable. Although it's hard to quantify abstract things like idea contributions, our AI companion was able to make valid criticisms, proper suggestions, good idea generation, and consistent formatting. Despite this, all of the finalized writing, full ideas, and architecture decisions were made by human group members. If AI was used, it was also heavily checked and discussed by the collective group, to

the point where most of it ended up being original anyway. All in all, despite our AI companion's help to help organize our group and provide feedback, it only contributed at most about 10% of our group's collective efforts.

Reflection on Human-AI Team Dynamics

Working with an AI teammate in Assignment 2 really reinforced our experience from Assignment 1 that AI works best as a supporting collaborator in comparison to a decision-maker. Our AI teammate helped a lot with idea generation and with structuring some of the early parts of the architecture analysis. It allowed us to quickly explore possible subsystem groupings and interpretations of the repository structure before we verified them ourselves. During this process we still had to complete careful validation and prompt refinement. By consistently including repository context, dependency information, and course materials in our prompts we were able to consistently avoid some responses that were occasionally overly confident or slightly off from what the system was doing.
Prompting the AI as a group also improved our internal communication. Before sending a prompt, we would briefly discuss what we were trying to figure out and how to phrase the question, which helped clarify our own understanding of the architecture. Through this process we realized that AI is most useful for brainstorming ideas and helping organize information, while the actual interpretation of the code and the final architectural decisions should always come from the human team members.

12. References

References

Google. (2025). *Gemini CLI documentation*. https://geminicli.com/docs/

Google. (2025). *Gemini CLI GitHub repository*. https://github.com/google-gemini/gemini-cli

Google. (2025). *Gemini CLI roadmap*. https://github.com/google-gemini/gemini-cli/blob/main/ROADMAP.md

Google. (2025). *Gemini CLI hands-on tutorial*. https://codelabs.developers.google.com/gemini-cli-hands-on

Jalateras. (2024). *Unpacking the Gemini CLI: A high-level architectural overview*. https://medium.com/@jalateras/unpacking-the-gemini-cli-a-high-level-architectural-overview-99212f6780e7

SciTools. (2024). *Understand static analysis tool documentation*. https://scitools.com/

Virtuslab Team. "How Claude Code Works: An Inside Look at the Textual UI Agent." *Virtuslab Blog*, https://virtuslab.com/blog/ai/how-claude-code-works/

Garlan, David, and Mary Shaw. *An Introduction to Software Architecture*. Carnegie Mellon University, Jan. 1994, CMU-CS-94-166.